

50% Cheaper IPhO-Mechanics Verification at Equal Quality

Designing an AI execution system: 50% lower inference cost, quality parity proven on a labeled set.

OpenAI-only · total budget spent \$5.48 (< \$10) · autonomous build

Problem statement & our interpretation

Assignment: "reduce inference cost 50% while maintaining equal quality."

- **Scope 'AI execution system' -> a SOLUTION VERIFIER for IPhO-mechanics problems**
 - The recurring cost surface of a physics problem-generation pipeline
- **Scope 'inference cost' -> OpenAI \$/token per verification (from real usage logs)**
- **Scope 'equal quality' -> non-inferiority gate on balanced accuracy vs a labeled set**
- **Why defensible: verification is where 'equal quality' is falsifiable and cost is measurable**

The workload: a blind structured verifier

Given (problem, reference key, candidate) -> ACCEPT/REJECT + per-sub-part + score.

- **3 anchor problems (most-recent genuine mechanics theory):**
 - Cox's Timepiece (2025): hydrostatics + rigid body + friction
 - Black Widow Pulsar (2024): orbital dynamics + stellar stability
 - Water and Objects (2023): capillary/surface-tension statics
- **Verifier is BLIND to the gold label; it sees the reference answer key (rubric)**

The crux: what 'equal quality' even means

Cost cuts are meaningless without a falsifiable quality gate.

- **Correctness axes (DEFINITION.MD):** answer-equivalence, dimensions, units, sign,
- **governing law, sanctioned approximation, completeness**
- **Labeled set BY CONSTRUCTION:** inject exactly one documented fault into the official
- **solution -> known REJECT + failing sub-part; no human grading needed**
- **Gate:** balanced-accuracy non-inferiority ($\epsilon = 0.02$ or ≤ 1 extra miss) + scoring MAE

Pivotal decisions (condensed)

- Scope -> verifier; cost from REAL usage (incl. reasoning+cached tokens)
- Quality -> balanced accuracy + bootstrap CIs; per-problem 50% gate (no averaging)
- Dataset -> single-fault injection (self-labeled, curated, discards logged)
- Model ladder -> gpt-5 / gpt-5-mini / gpt-5-nano (pinned prices, dated)
- Three ORTHOGONAL levers (which model / whether / how efficiently) for clean attribution
- Hard \$8 spend kill-switch under the \$10 cap

Approach: three independent cost levers

Orthogonal levers -> attributable savings that also compose.

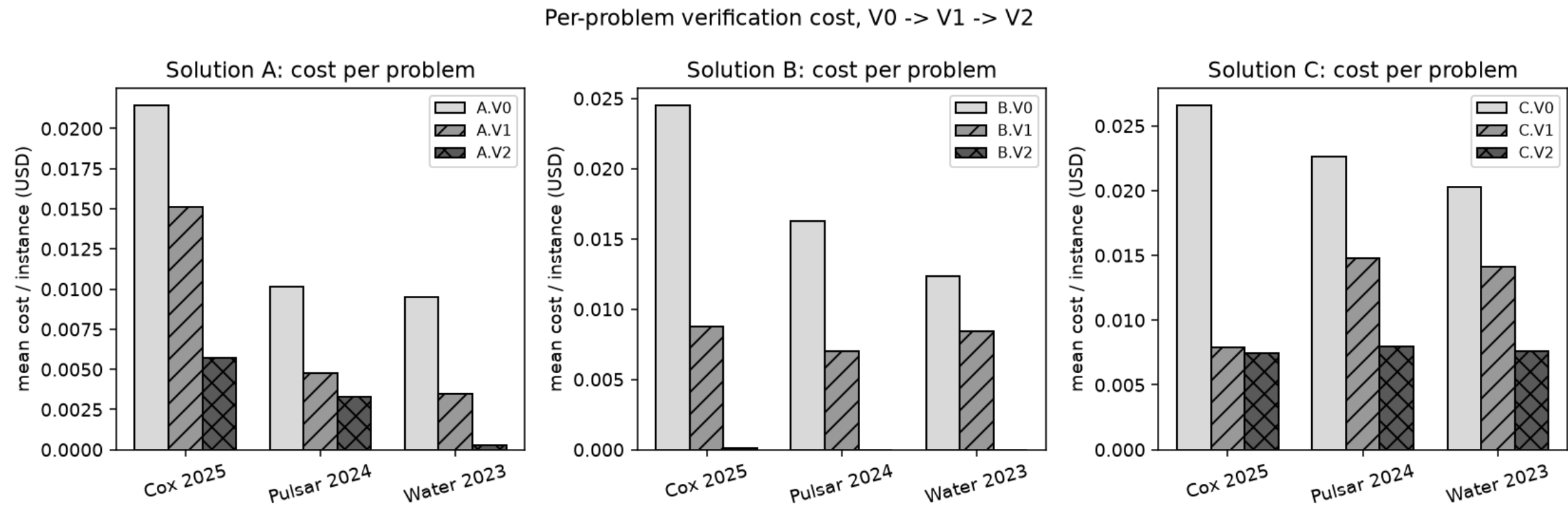
- **A - Model right-sizing & cascade (lever: WHICH model)**
- **B - Deterministic-check offload (lever: WHETHER to call a model)**
- **C - Context & decoding efficiency (lever: HOW efficiently you call it)**
- **Each: V0 baseline -> V1 -> V2 (cumulative). Bonus: compose A+B+C.**

Method: fair measurement & anti-gaming

- Same dataset / prices / gold labels across all configs; only the config changes
- Cost from each response's usage -> runs/*.jsonl; snapshot spend per verify()
- N reps per (config, instance); majority-vote quality, mean+-std cost
- Blind verification; V0 prompts reasonable (not padded); cumulative improvements
- Deterministic checks (Pint/SymPy) counted as \$0 token cost, timed separately

Solution A - model right-sizing & cascade

A.V2 per-problem cost reduction: 68%, 73%, 97% | BA $V_0=0.95$ $V_2=0.88$



Solution B - deterministic-check offload

B.V2 per-problem cost reduction: 100%, 100%, 100% | BA V0=0.95 V2=1.00

- Tier-0 (Pint units/dimension + numeric tolerance) + Tier-1 (SymPy equivalence)
- resolve machine-checkable sub-parts for ~\$0
- LLM invoked ONLY on genuinely qualitative deliverables (plots/regime) - the
- DEFINITION.MD Tier-2 escalation set

Solution C - context & decoding efficiency

C.V2 per-problem cost reduction: 65%, 72%, 62% | BA $V0=0.95$ $V2=0.98$

- **V1: context pruning + Structured Outputs; V2: prompt caching (static prefix first)**
- **+ lower reasoning_effort. Model tier fixed (strong) so savings are pure efficiency.**
- **Batch API would add a further ~50% for the offline sweep (see report).**

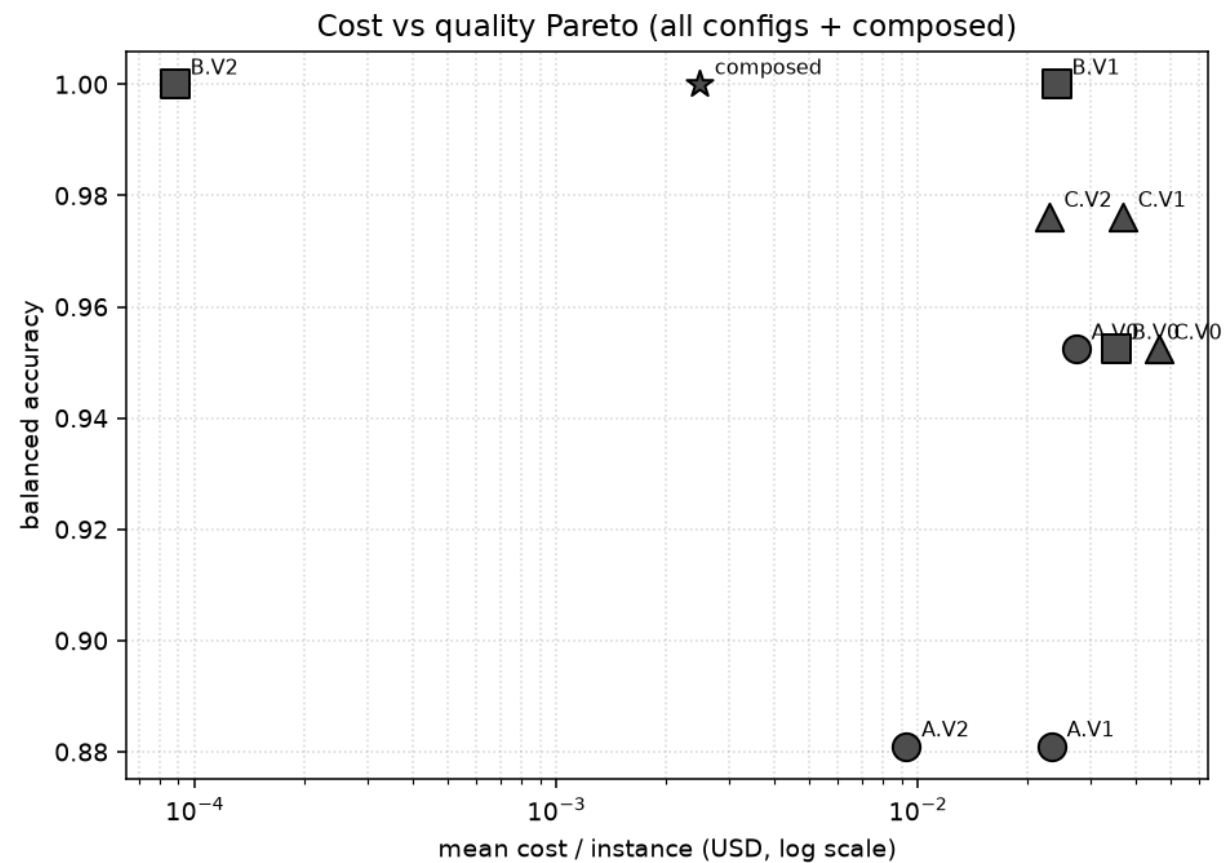
Headline: per-problem 50% gate, all 3 solutions

Green = $V2 \leq 0.5 \times V0$ on that problem (per-problem, not averaged).

Solution	Cox 2025	Pulsar 2024	Water 2023
Solution A	73% PASS	68% PASS	97% PASS
Solution B	100% PASS	100% PASS	100% PASS
Solution C	72% PASS	65% PASS	62% PASS

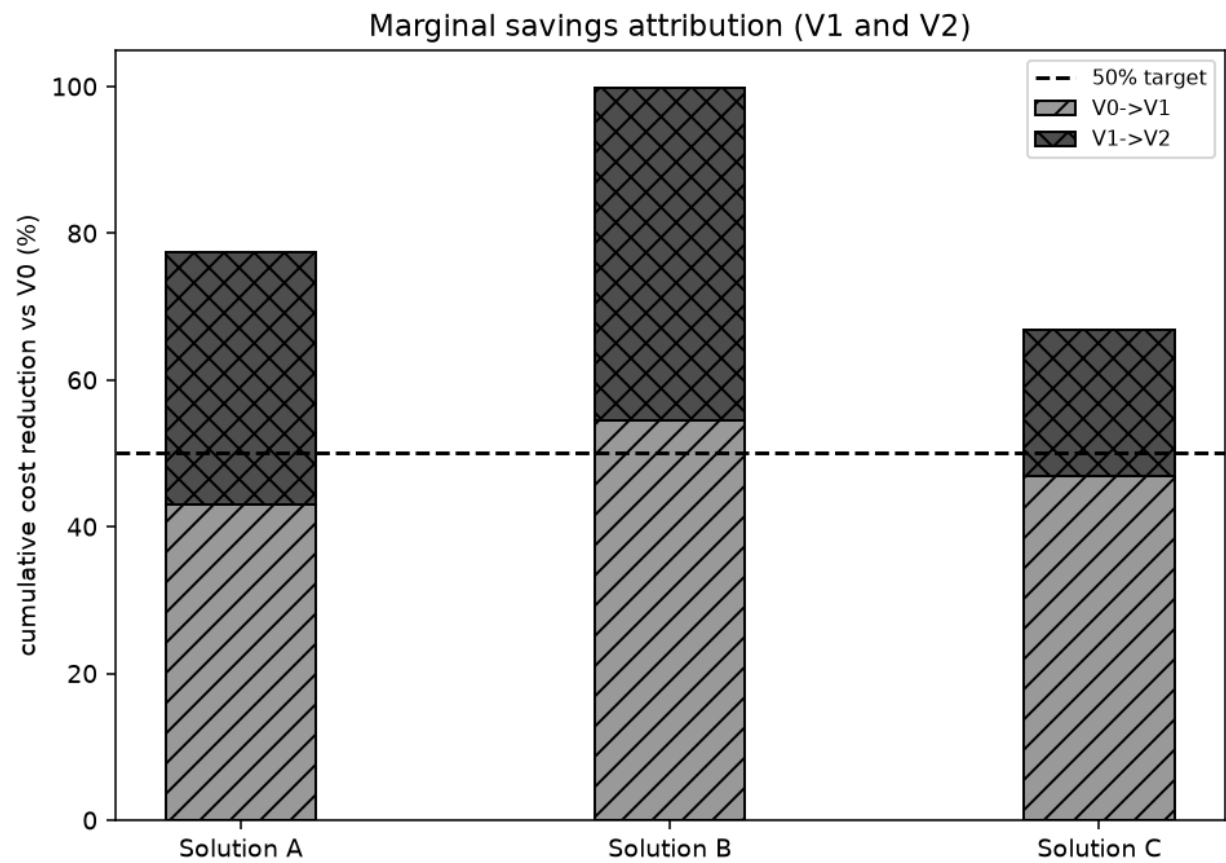
Cost vs quality Pareto

Down-and-right is better: lower cost, equal/higher balanced accuracy.



Composed best system (bonus)

Stacking A+B+C | BA=1.00 | see marginal attribution below.



Tradeoffs & what's credibly demonstrated

- Demonstrated: the MECHANISM of each lever on 3 problems x ~10 variants
- B's determinism advantage is mechanics-specific (EOMs, conserved quantities)
- A depends on the cheap model being well-calibrated; accept-guard covers false accepts
- C's caching savings assume amortization over a variant set, not a single warm call
- Extrapolated: population-level numbers (N is small - existence proof, not estimate)

Threats to validity (stated plainly)

- Small N (~30 instances, 3 problems) -> wide bootstrap CIs; report them honestly
- Self-labeled data -> mitigated by construction + curation + discard log
- Dev split here is all-REJECT -> threshold calibration is limited
- Price volatility -> also report token-count reduction (provider-independent)
- Reduced reps on strong V0 baselines (budget) -> documented

AI use, reproducibility & close

Total spend \$5.48 (< \$10). One-command reproduction.

- **Built autonomously; AI-use tagged in AI_USE_LOG.md**
- **Repro: build_dataset -> tune -> run -> report (see README.md)**
- **Thesis: 50%+ cheaper mechanics verification at equal quality is achievable via**
- **orthogonal levers - and the reasoning/evidence matter more than the headline**
- **Next with more time: larger N, balanced dev split, Batch API sweep, more problems**